

Specification of the Angelfish consensus algorithm at a high level of abstraction. We use a number of abstractions and simplifying assumptions in order to expose the high-level principles of the algorithm clearly.

EXTENDS *Integers, FiniteSets, Sequences*

CONSTANTS

N The set of all nodes
 R The set of rounds
 $IsQuorum(-)$ Whether a set is a quorum (*i.e.* cardinality at least $n-f$)
 $IsBlocking(-)$ Whether a set is a blocking set (*i.e.* cardinality at least $f+1$)
 $Leader(-)$ operator mapping each round to its leader

ASSUME $\exists n \in R : R = 1 \dots n$ useful rounds start at 1

See *BlockDag* definitions in *BlockDag.tla*:

INSTANCE *BlockDag* WITH $N \leftarrow N, R \leftarrow R, Leader \leftarrow Leader$

A vote by a node for a vertex or for no vertex ($\langle \rangle$):

$Vote \triangleq N \times R \times (V \cup \{\langle \rangle\})$

$Vertex(v) \triangleq v[3]$

Timeout messages:

$Timeout \triangleq N \times R$

Quorum of messages:

$IsQuorumMsgs(M) \triangleq IsQuorum(\{Node(m) : m \in M\})$

--algorithm *Angelfish*{

variables

$vs = \{Genesis\}$, the vertices of the *DAG*

$es = \{\}$; the strong edges of the *DAG* (we do not model weak edges)

$votes = \{\}$; vote messages

$timeouts = \{\}$; timeout messages

define {

$dag \triangleq \langle vs, es \rangle$

whether we have timeout certificates for all rounds from $Round(l)$ to $rnd - 1$:

$ValidPreviousLeader(l, rnd) \triangleq$

$\wedge IsLeader(l)$

$\wedge \forall r \in (Round(l) + 1) \dots (rnd - 1) :$

$IsQuorumMsgs(\{to \in timeouts : Round(to) = r\})$

whether a *DAG* d is valid:

$IsValid(d) \triangleq \forall v \in Vertices(d) : v \neq \langle \rangle \Rightarrow$

$\wedge Node(v) \in N \wedge Round(v) \in Nat \setminus \{0\}$

$\wedge \text{IF } \neg IsLeader(v)$

THEN $\forall v2 \in Children(d, v) : Round(v2) = Round(v) - 1$

```

ELSE  $\exists l \in \text{Children}(d, v) :$ 
     $\wedge \text{IsLeader}(l) \wedge \text{Round}(l) < \text{Round}(v)$ 
     $\wedge \text{ValidPreviousLeader}(l, \text{Round}(v))$ 
     $\wedge \forall v2 \in \text{Children}(d, v) :$ 
         $v2 \neq l \Rightarrow \text{Round}(v2) = \text{Round}(v) - 1$ 
}
macro StartRound1( ) {
    the first round is a bit special since there's no previous leader
    round := 1 ;
    either { create a vertex:
        vs := vs  $\cup$  { $\langle \text{self}, 1 \rangle$ } ;
        es := es  $\cup$  { $\langle \langle \text{self}, 1 \rangle, \text{Genesis} \rangle$ } edge to genesis
    }
    or or send an empty vote:
        votes := votes  $\cup$  { $\langle \text{self}, 1, \langle \rangle \rangle$ }
    }
macro Vote( deliveredVertices ) {
    if ( LeaderVertex(round)  $\in$  deliveredVertices  $\wedge$   $\langle \text{self}, \text{round} \rangle \notin \text{timeouts}$  )
        votes := votes  $\cup$  { $\langle \text{self}, \text{round} + 1, \text{LeaderVertex}(\text{round}) \rangle$ }
    else
        votes := votes  $\cup$  { $\langle \text{self}, \text{round} + 1, \langle \rangle \rangle$ }
    }
macro CreateVertex( receivedVertices ) {
    if ( LeaderVertex(round)  $\in$  receivedVertices  $\wedge$   $\langle \text{self}, \text{round} \rangle \notin \text{timeouts}$  ) {
        create a vertex pointing to the leader of the previous round
        with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
            vs := vs  $\cup$  {newV} ;
            es := es  $\cup$  { $\langle \text{newV}, pv \rangle : pv \in \text{receivedVertices}$ }
        }
    }
    else {
        if ( Leader(round + 1)  $\neq$  self )
            create a vertex not pointing to any leader
            with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
                vs := vs  $\cup$  {newV} ;
                es := es  $\cup$  { $\langle \text{newV}, pv \rangle : pv \in \text{deliveredVertices} \setminus \{ \text{LeaderVertex}(\text{round}) \}$ }
            }
        }
    }
    else
        we need to find a previous leader to point to
        with ( prevLeader  $\in$  { v  $\in$  vs :
            Round(v) < round  $\wedge$  v = LeaderVertex(Round(v)) } )
        with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
            we must have timeout certificates down to the previous leader:
            when ValidPreviousLeader(prevLeader, round + 1) ;
            vs := vs  $\cup$  {newV} ;
        }
    }
}

```

```

        es := es
              ∪ {⟨newV, pv⟩ : pv ∈ deliveredVertices \ {LeaderVertex(round)}}
              ∪ {⟨newV, prevLeader⟩};   point to the previous leader
    }
  }
}
macro TryCommitLeader( deliveredVertices, deliveredVotes ) {
  with ( l = LeaderVertex(round - 1) )
  with ( support = vertices pointing to l plus votes
        {v ∈ deliveredVertices : ⟨v, l⟩ ∈ es} ∪ {vt ∈ deliveredVotes : vt[3] = l} )
  if ( IsQuorumMsgs(support) )
    log := Linearize(SubDag(dag, {l}), l);
  }
macro Catchup( ) {
  catch up to a round r strictly greater than round + 1
  with ( M ∈ SUBSET ((vs ∪ votes ∪ timeouts) \ {Genesis}) \ {[]} )
  with ( r = Min({Round(m) : m ∈ M}) )
  with ( B = {Node(m) : m ∈ M} ) {
    when r > round + 1 ∧ IsBlocking(B);
    round := r;
  }
}

```

Finally, we give the full specification of a node:

```

process ( correctNode ∈ N )
  variables
    round = 0,   current round; 0 means the node has not started execution
    log = ⟨⟩;   delivered log
  {
l0:   StartRound1();
l1:   while ( TRUE )
      either
        start the next round and create a vertex or vote
        with ( deliveredVertices ∈ SUBSET {v ∈ vs \ {⟨⟩} : Round(v) = round} )
        with ( deliveredVotes ∈ SUBSET {vt ∈ votes \ {⟨⟩} : Round(vt) = round} )
        with ( deliveredTimeouts ∈ SUBSET {to ∈ timeouts \ {⟨⟩} : Round(to) = round} ) {
          we heard from a quorum:
          when IsQuorumMsgs(deliveredVertices ∪ deliveredVotes ∪ deliveredTimeouts);
          we have the leader or a timeout certificate:
          when LeaderVertex(round) ∈ deliveredVertices ∨ IsQuorumMsgs(deliveredTimeouts);
          either {
            when (Leader(round + 1) ≠ self);   leaders cannot vote
            Vote(deliveredVertices)
          }
        }
      or

```

```

        CreateVertex(deliveredVertices);
        possibly commit the leader of round - 1:
    if ( round > 1 )
        TryCommitLeader(deliveredVertices, deliveredVotes);
        finally, increment the round counter:
        round := round + 1
    }
    or {
        if we have the leader, we can also vote without having heard from a quorum:
        when (Leader(round + 1) ≠ self);  leaders cannot vote
        when (Leader(round) ∈ vs);
        Vote({Leader(round)})
    }
    or
        timeout (this means we cannot support the leader of this round in the next round)
        timeouts := timeouts ∪ {⟨self, round⟩}
    or
        Catchup()
}
}
BEGIN TRANSLATION (chksum(pcal) = "9fea5d2d" ∧ chksum(tla) = "f4ef0e95")
VARIABLES pc, vs, es, votes, timeouts

define statement
dag ≜ ⟨vs, es⟩

ValidPreviousLeader(l, rnd) ≜
    ∧ IsLeader(l)
    ∧ ∀ r ∈ (Round(l) + 1) .. (rnd - 1) :
        IsQuorumMsgs({to ∈ timeouts : Round(to) = r})

IsValid(d) ≜ ∀ v ∈ Vertices(d) : v ≠ ⟨⟩ ⇒
    ∧ Node(v) ∈ N ∧ Round(v) ∈ Nat \ {0}
    ∧ IF ¬IsLeader(v)
        THEN ∀ v2 ∈ Children(d, v) : Round(v2) = Round(v) - 1
    ELSE ∃ l ∈ Children(d, v) :
        ∧ IsLeader(l) ∧ Round(l) < Round(v)
        ∧ ValidPreviousLeader(l, Round(v))
        ∧ ∀ v2 ∈ Children(d, v) :
            v2 ≠ l ⇒ Round(v2) = Round(v) - 1

VARIABLES round, log

vars ≜ ⟨pc, vs, es, votes, timeouts, round, log⟩

ProcSet ≜ (N)

```

$$\begin{aligned}
Init &\triangleq \text{Global variables} \\
&\wedge vs = \{Genesis\} \\
&\wedge es = \{\} \\
&\wedge votes = \{\} \\
&\wedge timeouts = \{\} \\
&\text{Process } correctNode \\
&\wedge round = [self \in N \mapsto 0] \\
&\wedge log = [self \in N \mapsto \langle \rangle] \\
&\wedge pc = [self \in ProcSet \mapsto "I0"] \\
l0(self) &\triangleq \wedge pc[self] = "I0" \\
&\wedge round' = [round \text{ EXCEPT } ![self] = 1] \\
&\wedge \vee \wedge vs' = (vs \cup \{\langle self, 1 \rangle\}) \\
&\quad \wedge es' = (es \cup \{\langle \langle self, 1 \rangle, Genesis \rangle\}) \\
&\quad \wedge votes' = votes \\
&\quad \vee \wedge votes' = (votes \cup \{\langle self, 1, \langle \rangle \rangle\}) \\
&\quad \wedge \text{UNCHANGED } \langle vs, es \rangle \\
&\wedge pc' = [pc \text{ EXCEPT } ![self] = "I1"] \\
&\wedge \text{UNCHANGED } \langle timeouts, log \rangle \\
l1(self) &\triangleq \wedge pc[self] = "I1" \\
&\wedge \vee \wedge \exists deliveredVertices \in \text{SUBSET } \{v \in vs \setminus \{\langle \rangle\} : Round(v) = round[self]\} : \\
&\quad \exists deliveredVotes \in \text{SUBSET } \{vt \in votes \setminus \{\langle \rangle\} : Round(vt) = round[self]\} : \\
&\quad \exists deliveredTimeouts \in \text{SUBSET } \{to \in timeouts \setminus \{\langle \rangle\} : Round(to) = round[self]\} : \\
&\quad \wedge IsQuorumMsgs(deliveredVertices \cup deliveredVotes \cup deliveredTimeouts) \\
&\quad \wedge LeaderVertex(round[self]) \in deliveredVertices \vee IsQuorumMsgs(deliveredTimeouts) \\
&\quad \wedge \vee \wedge (Leader(round[self] + 1) \neq self) \\
&\quad \quad \wedge \text{IF } LeaderVertex(round[self]) \in deliveredVertices \wedge \langle self, round[self] \rangle \notin deliveredVertices \\
&\quad \quad \quad \text{THEN } \wedge votes' = (votes \cup \{\langle self, round[self] + 1, LeaderVertex(round[self]) \rangle\}) \\
&\quad \quad \quad \text{ELSE } \wedge votes' = (votes \cup \{\langle self, round[self] + 1, \langle \rangle \rangle\}) \\
&\quad \quad \wedge \text{UNCHANGED } \langle vs, es \rangle \\
&\quad \vee \wedge \text{IF } LeaderVertex(round[self]) \in deliveredVertices \wedge \langle self, round[self] \rangle \notin deliveredVertices \\
&\quad \quad \text{THEN } \wedge \text{LET } newV \triangleq \langle self, round[self] + 1 \rangle \text{IN} \\
&\quad \quad \quad \wedge vs' = (vs \cup \{newV\}) \\
&\quad \quad \quad \wedge es' = (es \cup \{\langle newV, pv \rangle : pv \in deliveredVertices\}) \\
&\quad \text{ELSE } \wedge \text{IF } Leader(round[self] + 1) \neq self \\
&\quad \quad \text{THEN } \wedge \text{LET } newV \triangleq \langle self, round[self] + 1 \rangle \text{IN} \\
&\quad \quad \quad \wedge vs' = (vs \cup \{newV\}) \\
&\quad \quad \quad \wedge es' = (es \cup \{\langle newV, pv \rangle : pv \in deliveredVertices\}) \\
&\quad \text{ELSE } \wedge \exists prevLeader \in \{v \in vs : Round(v) < round[self] \wedge v = Leader(v)\} \\
&\quad \quad \text{LET } newV \triangleq \langle self, round[self] + 1 \rangle \text{IN} \\
&\quad \quad \quad \wedge ValidPreviousLeader(prevLeader, round[self]) \\
&\quad \quad \quad \wedge vs' = (vs \cup \{newV\}) \\
&\quad \quad \quad \wedge es' = es
\end{aligned}$$

$$\begin{aligned}
& \cup \{\langle newV, pv \rangle : pv \in deliveredVertices\} \\
& \cup \{\langle newV, prevLeader \rangle\} \\
& \wedge votes' = votes \\
& \wedge \text{IF } round[self] > 1 \\
& \quad \text{THEN } \wedge \text{LET } l \triangleq LeaderVertex(round[self] - 1) \text{ IN} \\
& \quad \quad \text{LET } support \triangleq \{v \in deliveredVertices : \langle v, l \rangle \in es'\} \cup \{vt \in deliveredVertices : \langle vt, l \rangle \in es'\} \\
& \quad \quad \text{IF } IsQuorumMsgs(support) \\
& \quad \quad \quad \text{THEN } \wedge log' = [log \text{ EXCEPT } ![self] = Linearize(SubDag(dag, support))] \\
& \quad \quad \quad \text{ELSE } \wedge \text{TRUE} \\
& \quad \quad \quad \wedge log' = log \\
& \quad \text{ELSE } \wedge \text{TRUE} \\
& \quad \quad \wedge log' = log \\
& \quad \wedge round' = [round \text{ EXCEPT } ![self] = round[self] + 1] \\
& \quad \wedge \text{UNCHANGED } timeouts \\
& \vee \wedge (Leader(round[self] + 1) \neq self) \\
& \quad \wedge (Leader(round[self]) \in vs) \\
& \quad \wedge \text{IF } LeaderVertex(round[self]) \in (\{Leader(round[self])\}) \wedge \langle self, round[self] \rangle \notin timeouts \\
& \quad \quad \text{THEN } \wedge votes' = (votes \cup \{\langle self, round[self] + 1, LeaderVertex(round[self]) \rangle\}) \\
& \quad \quad \text{ELSE } \wedge votes' = (votes \cup \{\langle self, round[self] + 1, \rangle\}) \\
& \quad \wedge \text{UNCHANGED } \langle vs, es, timeouts, round, log \rangle \\
& \vee \wedge timeouts' = (timeouts \cup \{\langle self, round[self] \rangle\}) \\
& \quad \wedge \text{UNCHANGED } \langle vs, es, votes, round, log \rangle \\
& \vee \wedge \exists M \in \text{SUBSET}((vs \cup votes \cup timeouts) \setminus \{Genesis\}) \setminus \{\{\}\} : \\
& \quad \quad \text{LET } r \triangleq Min(\{Round(m) : m \in M\}) \text{ IN} \\
& \quad \quad \text{LET } B \triangleq \{Node(m) : m \in M\} \text{ IN} \\
& \quad \quad \quad \wedge r > round[self] + 1 \wedge IsBlocking(B) \\
& \quad \quad \quad \wedge round' = [round \text{ EXCEPT } ![self] = r] \\
& \quad \quad \wedge \text{UNCHANGED } \langle vs, es, votes, timeouts, log \rangle \\
& \wedge pc' = [pc \text{ EXCEPT } ![self] = "l1"]
\end{aligned}$$

$$correctNode(self) \triangleq l0(self) \vee l1(self)$$

Allow infinite stuttering to prevent deadlock on termination.

$$Terminating \triangleq \wedge \forall self \in ProcSet : pc[self] = \text{"Done"}$$

$$\wedge \text{UNCHANGED } vars$$

$$Next \triangleq (\exists self \in N : correctNode(self))$$

$$\vee Terminating$$

$$Spec \triangleq Init \wedge \Box [Next]_{vars}$$

$$Termination \triangleq \Diamond (\forall self \in ProcSet : pc[self] = \text{"Done"})$$

END TRANSLATION

Correctness properties:

